

1 All Sorts Of Sorts

Show the steps taken by each sort on the following unordered list:

0, 4, 2, 7, 6, 1, 3, 5

6, 1, 2, 3, 4, 5, 6, 7

(a) Insertion sort

0

0,4

0, 2, 4

0, 2, 4, 7

0, 2, 4, 6, ...

0, 1, 2, 4, 6, 7

0, 1, 2, 3, 4, 6, 7

(b) Selection sort

0

0 1 2 7 6 4 3 5

$$0 \overline{) 276435}$$

0 1 ¹2 3 6 4 7 5

0 1 2 3 4 6 7 5

0 1 2 3 4 5 7 6

0 1 2 3 4 5 6 7
Menge gest

(c) Merge sort

down-top

0 4 2 7 6 1 3 5

0 4 27 6 1 35

$$\begin{array}{r} 0 \ 4 \ \underline{27} \ 6 \ 1 \ 35 \\ 24 \end{array}$$

0 2 4 7 5 1 3 5

$$\underline{02471635}$$

0 2 4 7 16 25

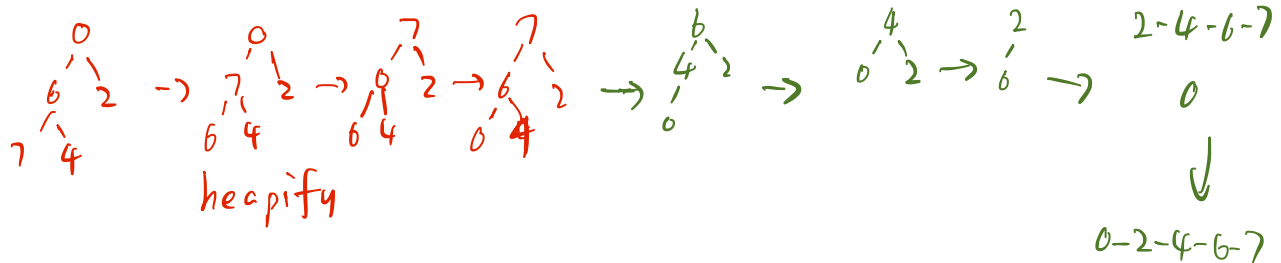
0 2 4 7 1 3 5 6

0 1 2 3 4 5 6 7

0 1 2 3 4 5 6 7

(d) Use heapsort to sort the following array (hint: draw out the heap). Draw out the array at each step:

0, 6, 2, 7, 4



2 Sorta Interesting, Right?

- (a) What does it mean to sort "in place", and why would we want this?

use extra constant space/memory

- (b) What does it mean for a sort to be "stable"? Which sorting algorithms that we have seen are stable?

stable means equivalent element's order won't change in sorting.

- (c) Which algorithm would run the fastest on an already sorted list?

Insertion Sort

- (d) Given any list, what is the ideal pivot for quicksort?

median. Split list in half will be most efficient.

- (e) So far, in class, we've mostly applied our sorts to lists of numbers. In practice, how would we typically make sure our sorts can be applied to other types?

For java It is comparable interface.

3 Zero One Two-Step

- (a) Given an array that only contains 0's, 1's and 2's, write an algorithm to sort it in linear time. You may want to use the provided helper method, `swap`.

```
public static int[] specialSort(int[] arr) {
    int front = 0;
    int back = arr.length - 1;
    int curr = 0;
    while (curr <= back) {
        if (arr[curr] == 0) {
            swap(arr, curr, front);
            front += 1;
            curr += 1;
        } else if (arr[curr] == 1) {
            curr += 1;
        } else {
            swap(arr, curr, back);
            back -= 1;
        }
    }
}
```

```
private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

- (b) We just wrote a linear time sort, how cool! Can you explain in a sentence or two why we can't always use this sort, even though it has better runtime than Mergesort or Quicksort?

This case only has three values..